

Event annotation reference — CNeuroMod videogame datasets

Every event type emitted into `sub-*/ses-*/func/*_desc-annotated_events.tsv`, what it means, and exactly which RAM variables and values produce it.

Generated by `videogames_utils.events`; the machine-readable version of the vocabulary is each dataset's `task-<task>_events.json`.

dataset	game	console	event rows	distinct trial_type	repetitions
mario	Super Mario Bros.	NES	599,061	73	3,374
mario3	Super Mario Bros. 3	NES	335,516	221	4,063
mariostars	Super Mario All-Stars (SMB1)	SNES	226,633	73	1,232
shinobi	Shinobi III	Genesis	244,032	24	666

`mario3` has many more distinct types only because SMB3 has ~170 named object types, and both `Enemy_on_screen/{type}` and `Enemy_defeated/{method}/{type}` expand one per species. Counts are from the files regenerated on 2026-09-12 (`shinobi` before its `hit_timer` replay pass, so without `Player_state/Hit_recovery`).

Each dataset's `code/annotations/README.md` is generated from the same vocabulary (`python -m videogames_utils.events.docs <dataset> <task>`) and lists, for that game, every entry it may emit with its row count, naming the entries that are defined but never produced and why. This document is the cross-game reference behind those.

How to read this document

Event names are hierarchical: the part before the `/` is the family, the part after is a qualifier. `{type}` in a heading means the suffix is filled in at generation time with a decoded object name, e.g. `Enemy_on_screen/Goomba`.

Columns

Column	Meaning
<code>trial_type</code>	The event, from the controlled vocabulary below.
<code>level</code>	Level of the repetition the event belongs to.
<code>onset</code>	Seconds from the start of the fMRI run.
<code>duration</code>	Seconds; 0 for point events.
<code>frame_start</code> , <code>frame_stop</code>	Emulator frames, relative to the repetition.
<code>button</code>	Raw controller button behind an <code>Action/*</code> event; empty otherwise.
<code>stim_file</code>	Path to the <code>.bk2</code> . Set on the gym-retro_game row only — every other event sits inside exactly one of those windows.

Per-repetition metadata — `phase`, `IndexInRun`, `IndexGlobal`, `IndexLevel` and `Outcome` — is **not** repeated on event rows. It lives in the repetition's `gamelogs/*_summary.json`, keyed by the `stim_file` on the container row.

Onsets are seconds from the start of the fMRI run, computed at the console's true frame rate — **60.099827 Hz** (NES), **60.098812 Hz** (SNES), **59.922743 Hz** (Genesis) — read from the emulator core rather than assumed to be 60.

The engine state variable

Most Super Mario Bros. events are read from one variable, the game's own statement of what the player is doing. It is GameEngineSubroutine at \$000E, exposed as `player_state` on the NES and `player_action_state` on the SNES (Super Mario All-Stars reuses the same enumeration). Values, from the disassembly's dispatch table:

value	routine	used for
0	Entrance_GameTimerSetup	Screen/Level_intro (the black title card, at a level start or after a life loss) or Screen/Transition (the ~24-frame black screen between areas)
1	Vine_AutoClimb	Screen/Transition
2	SideExitPipeEntry	Pipe_entered, Screen/Transition
3	VerticalPipeEntry	Pipe_entered, Screen/Transition
4	FlagpoleSlide	Level_completed (fallback), start of Screen/Level_end
5	PlayerEndLevel	Level_completed, Screen/Level_end
6	PlayerLoseLife	Player_died/Fall (end of; see the fall-timing note), end of Screen/Death
7	PlayerEntrance	Screen/Transition (Mario emerging from a pipe or dropping into the area)
8	PlayerCtrlRoutine	normal play, Screen/Gameplay
9	PlayerChangeSize	start of Player_state/Super (the status byte changes on this frame)
10	PlayerInjuryBlink	Player_damaged; the form row ends and Player_state/Hit_recovery starts
11	PlayerDeath	Player_died/Enemy, start of Screen/Death
12	PlayerFireFlower	start of Player_state/Fire

Using the engine rather than counters matters because counters lag: lives only drops once the death animation has finished, several seconds after the death itself.

On-screen visibility

Enemy_on_screen and friends are **screen-visibility** events: they start when the object becomes visible and their duration runs until it leaves the screen or is defeated. An object that leaves and returns produces two events. The predicate differs per game and each was validated against independently computed screen geometry:

The distinction being drawn is **on screen**, not **loaded in RAM**. Slot occupancy alone (Enemy_Flag, Objects_State) means only that the engine is simulating the object; all three games keep objects live well outside the camera, so occupancy is never used on its own except where noted below.

game	predicate	validation
mario	Enemy_Flag set and EnemyOffscrBitsMasked == 0	horizontally inside the visible band on 97.5–98.9% of frames when 0, 0.0–1.5% when non-zero; enemy_y inside 0–239 on 99.9% of accepted frames, so no separate vertical test is needed
mariostars	bit 0 of sprite_onscreen_flag_*	no per-sprite coordinates exist in this port, so the predicate is checked indirectly by cross-port agreement with the NES on the 21 shared levels
mario3	Objects_State != 0 and Objects_SprHVis == 0 and Objects_SprVVis == 0	SprHVis == 0 is horizontally inside the band on 99.9% of frames (n=9571) vs 11.7% when non-zero; among frames passing that test, SprVVis == 0 is vertically inside the band on 100.0% (n=37436) vs 5.1% when non-zero

SMB3 is the game where the two notions diverge most: it scrolls vertically, so a slot can be horizontally on screen while sitting entirely above or below it. The vertical flag removes 21% of the frames the horizontal test alone would have called visible.

Two deliberate exceptions use slot occupancy on its own, and mean "spawned" rather than "on screen": SMB1's Flagpole_visible / Castle_visible (scenery never clears the enemy offscreen test — 0 of 479 frames in a sampled replay), and any repetition whose _variables.json predates the visibility addresses. shinobi has no object visibility at all; its Enemy_defeated is inferred from score deltas.

Tracks shorter than 5 frames are dropped as slot churn, and gaps of up to 4 frames are bridged, because a one-frame flicker of the slot bookkeeping is not a disappearance the player perceives.

Screens: what the game shows

The Screen/* rows say what the game is showing, as opposed to what the player is doing. They **partition every repetition**: the rows of a repetition are contiguous, do not overlap, and run from its first frame to its last, so exactly one is active at any frame and Screen/Gameplay is the complement of the rest. All are durational.

Row	Meaning	mario / mariostars	mario3	shinobi
Screen/Gameplay	The player controls the character in the level.	engine 8, 9, 10, 12	everything not below	blackScreen 0 or 40 (40 is the scroll lock of an enemy wave, not a black screen)
Screen/Level_intro	The level's title card.	engine 0 (NES) / OperMode_Task $\neq$ 3 (SNES) at the start of the repetition or directly after a death: the black "WORLD x-y / Mario $\times$ n" card, 86-193 frames	— (a repetition opens in the level with a short fade-in)	—
Screen/Death	The death sequence, no control.	engine 11 through 6, or from the fall (see below) to 6	player_is_dying $\neq$ 0, ~200 frames	from the frame health reaches 0 to the life loss: 127-344 frames of animation and fade
Screen/Level_end	The end-of-level sequence, no control.	engine 4 then 5: flagpole slide, walk into the castle, time bonus, 6-14 s	from the goal card to the end of the recording: card animation and COURSE CLEAR, ~340 frames	the end-of-level fade running to the end of the recording (blackScreen 22 or 62): ~380 frames of ROUND CLEAR tally on level 1, 15-16 frames on levels 4 and 5
Screen/Transition	Between areas, no control.	engine 1, 2, 3, 7 and the 0 / task $\neq$ 3 stretch between them: pipe travel, vine climb, the exit from a bonus area, 24 frames of black	player_halt_game $\geq$ 128 while the next area loads, plus the pipe-entry hold before it	section fades (22 or 62 then 20, ~80 frames) and the fade back in after a death (21 then 20, ~85 frames)
Screen/Game_over	The GAME OVER screen.	— (recordings end at the life loss)	level_music = 0 with lives < 0 after the death: the map with the GAME OVER dialog. 148 of 4063 recordings ran on into this instead of stopping at the death, for a median 18 s and up to 4 min.	blackScreen 21 with lives < 0, 3 recordings
Screen/Map	The SMB3 world map.	—	level_music = 0 with lives $\geq$ 0 after the death: one recording (sub-06 ses-010 w6l4 rep-000) returned to the map and recorded a second attempt at the level	—

Every boundary was checked on rendered emulator frames. The SNES title card needs `OpMode_Task` (\$0772), which the engine variable cannot see (it idles at 8 while the card is up); it is read out of the shipped reset variable, a 4-byte little-endian BCD read at \$0771 whose second byte pair is that task (1 = card, 3 = game routines). The SNES fades in and out around the card, so those boundaries are within ~30 frames (0.5 s).

The `Player_state` **form rows** (Small, Super, Fire and the mario3 suits; Normal in shinobi) tile the frames on which the player is *alive in the level*: every frame not inside a `Screen/Death`, `Screen/Level_intro`, `Screen/Game_over` or `Screen/Map` row. Together, the two families give a form for every frame of play and a screen for every frame of the recording.

Timing a fall death

`PlayerLoseLife` does not run when the player drops into a pit — it runs about **4 s later**, once the game has let the player fall out of the level and is ready to reset. Recording the death there puts it seconds after anything the player saw, so onset is walked back to the frame the player actually left the screen and duration runs from there to the life loss. The old timing is still available as `onset + duration`.

game	how the fall is dated	accuracy
mario	<code>Player_Y_HighPos</code> (<code>player_y_screen</code>) leaving 1 — it is 1 on 95.0% of play frames, 0 above the screen, >1 below it	exact; 69 of 69 sampled fall deaths have the crossing, at a lag of 244–257 frames
mariostars	the frame the level timer stops ticking — the game freezes gameplay the moment the player drops out — plus half a tick	validated by running the same proxy on the NES against the exact crossing: median error –12 frames, 100% within 24 frames (0.4 s) ; the half-tick correction removes that bias
mario3	unchanged: <code>player_is_dying == 2</code> already fires at the drop, 210 frames (3.5 s) <i>before</i> the life is lost	the game's own flag

The SNES port exposes no player-Y address, which is why it needs the proxy. The `scroll_x_low` freeze was also tested as a proxy and is worse (only 77% within 0.4 s, p10 –50 frames), so the timer is used.

1. Super Mario Bros. — `mario` (NES) and `mariostars` (SNES)

Both ports share one implementation: Super Mario All-Stars reuses SMB1's object id table verbatim (verified empirically) and the same engine enumeration. Where the two differ, both variable names are given as *NES / SNES*.

Player

Event	Description	Detection
<code>Player_damaged</code>	Hit, losing the current power-up state.	<code>player_state</code> / <code>player_action_state</code> transitions to 10 (<code>PlayerInjuryBlink</code>).
<code>Player_died/Enemy</code>	Killed by an enemy or other damaging object.	Engine transitions to 11 (<code>PlayerDeath</code>).
<code>Player_died/Fall</code>	Fell into a pit.	Engine transitions to 6 (<code>PlayerLoseLife</code>) with no 11 in the preceding 400 frames — a fall bypasses the death routine. onset is walked back to the frame the player left the screen (see below); duration runs to the life loss.
<code>Player_died/Timeout</code>	The level timer ran out.	Engine reaches 11 while <code>time / time_units</code> has been 0 for the preceding 120 frames.
<code>Life_gained</code>	Gained an extra life.	lives increases.

Event	Description	Detection
Player_state/Small, Player_state/Super, Player_state/Fire	The form the player is in, one row per continuous stretch. Together they tile the frames on which the player is alive in the level.	Maximal runs of player_status == 0 / 1 / 2 over the alive frames (outside Screen/Death and Screen/Level_intro). PlayerStatus is written on the very frame the item is collected (engine 9 / 12) and on the hit frame (engine 10), so the row starts and ends on those frames. A row is cut at a death and Small resumes once the title card has gone.
Player_state/Star	Star invincibility, whole period.	Non-zero run of star_timer (StarInvincibleTimer, \$079F / \$07AF): 35 → 0 on the 21-frame interval clock, ~730 frames.
Player_state/Hit_recovery	Post-hit invulnerability: the ~3.7 s after a hit during which InjurePlayer ignores enemy contact and the player blinks.	Non-zero run of injury_timer (InjuryTimer, \$079E / \$07AE, new): 8 → 0 on the interval clock, paused during the shrink animation.

On the SNES, player_status, injury_timer and star_timer are new in this release (\$7E0756, \$7E07AE, \$7E07AF). The shipped player_powerup (\$0578) and star_power_timer (\$0553) hold unrelated bytes (values like 82 and 107, constants rather than a countdown) and are no longer read; see ram.SMAS_CANDIDATES for how the real ones were found.

player_status is a variable added in this release, reading PlayerStatus at \$0756 as a single byte. The pre-existing powerstate reads the same address as 4-byte BCD, which mixes in the neighbouring death flag and reports values like 10000. Use player_status.

Items and blocks

Event	Description	Detection
Item_collected/Coin	Collected a coin.	coins / player_coins increases, or drops by ≥99 (the counter wraps at 100 and awards a life).
Item_on_screen/Powerup	A mushroom, flower or star became visible.	An enemy slot holds object id 0x2E (PowerUpObject) and is visible.
Block_smashed	Destroyed a brick from below.	score increases by exactly 5 (NES, BCD in units of 10 points) or 50 (SNES, actual points) while the player is airborne (jump_airborne ∈ {1,2} on the NES), before the level ends. The end-of-level time bonus produces the same increment, so scoring stops at the level end.

Enemies

Event	Description	Detection
Enemy_on_screen/{type}	An enemy became visible.	A visible track opens in an object slot. Type from enemy_id_* / sprite_number_* decoded through the SMB1 object table. duration spans the visible lifetime.
Enemy_defeated/Stomp/{type}	Defeated by jumping on it.	Enemy_State / sprite_state_* transitions into 4 from a non-kill state.
Enemy_defeated/Projectile/{type}	Defeated by a fireball or similar.	The same array transitions into 34 .
Enemy_defeated/Shell/{type}	Defeated by a moving shell.	The same array transitions into 132 .
Projectile_on_screen/{type}	A Bowser flame is on screen.	A visible track whose object id is 0x15 (BowserFlame). Bullet Bills are not here — see below.

The defeat is emitted at the **first** frame of the kill state, i.e. when the enemy is hit — not when its corpse finishes falling. This matters because on a projectile kill the game deliberately moves the dying enemy's bounding box off screen, so reading the end of the animation would place the event after the enemy stopped being visible.

A defeat names both the method and the species, e.g. Enemy_defeated/Stomp/Goomba.

Object ids that are not enemies and are excluded from Enemy_*: moving platforms and lifts (0x24–0x2C), items and scenery (0x16, 0x2E, 0x2F, 0x30, 0x31, 0x32, 0x35, 0x36), spawner/control objects (0x17, 0x18) and the Bowser flame (0x15).

Bullet Bills are enemies, and live under two ids

A Bullet Bill can be stomped for points and kills the player on contact, so it is an enemy rather than a projectile. It occupies **two** object ids depending on which spawner made it — 0x08 from a frenzy, 0x33 from a cannon — and the disassembly names both after the spawner rather than after the object. Both now decode to BulletBill.

0x33 used to be classed as a spawner and discarded outright, which threw away most of the Bullet Bills in both datasets: over 120 mario replays it accounts for **331** flying runs against 90 for 0x08 (mariostars: 283 against 43). Runs of the two ids are indistinguishable in motion — both travel at a median **1.5 px/frame**. On one sampled session this took BulletBill from 25 tracks to 267.

The id is shared with the stationary cannon that fires them. On the NES, where per-slot X is available, a track is kept only if it moves in X on at least half its frames; 90% of visible tracks pass. The SNES port exposes no per-sprite X, so the test cannot run there and roughly 10% of its Enemy_on_screen/BulletBill tracks are expected to be the cannon.

Piranha Plants count only while out of the pipe

A Piranha Plant holds its slot for the whole cycle — rising, waiting, retracting, then sitting inside the pipe — and the offscreen bits describe only the horizontal edges, so the slot reads as visible even while the plant is hidden and cannot touch the player.

Its screen Y traces the cycle exactly: over 60 mario replays the plant moves through 25 distinct Y values spanning **24 px**, resting at the maximum. Within each run of the slot the resting position is that maximum, and the plant is out whenever it sits above it. A run that never shows the full 24 px cannot identify its own resting position and is left visible throughout rather than guessed at.

34.2% of what used to count as visible Piranha Plant time was the plant hidden in its pipe. One long track per pipe becomes one event per emergence: on a sampled session, 141 tracks of median 3.06 s became 228 of median 1.72 s.

Enemy_State was checked first and is **0** for the entire cycle, so it cannot serve — which is why the SNES port, exposing no per-sprite Y, keeps the old behaviour.

Note that **id 0x00 is Green Koopa**, a real enemy — a zero id does not mean an empty slot. Slot occupancy is given by Enemy_Flag, not by the id.

Environment and level

Event	Description	Detection
Pipe_entered	Entered a pipe.	Engine transitions to 2 (SideExitPipeEntry) or 3 (VerticalPipeEntry).
Flagpole_visible	The end-of-level flagpole came on screen.	An object slot holds id 0x30 (FlagpoleFlagObject). Judged by slot occupancy alone, because scenery never clears the enemy offscreen test.

Event	Description	Detection
Castle_visible	The end-of-level star flag / castle came on screen.	The same, with id 0x31 (StarFlagObject).
Timer_warning_started	The game began warning that time is short.	NES: bit 0x40 of event_music_queue (the game's own "time running out" music cue). SNES: the timer crossing 100 downward during normal play.
Level_started	A gameplay attempt began.	Frame 0 of the repetition.
Level_completed	The level was finished.	Engine transitions to 5 (PlayerEndLevel), falling back to 4 (FlagpoleSlide). Using PlayerEndLevel means this also fires on castle levels, which end at an axe rather than a flagpole.
Level_exited/Warp	The player left through a warp-zone pipe.	The world / area index changes and the level never ended. Only fires on w1-2 and w4-2, the two warp-zone levels.

Checkpoint_reached is deliberately not emitted. Its only candidate signal, HalfwayPage (\$075B), is written when the player *dies* past the midpoint in order to place the respawn — not when the midpoint is crossed. In a sampled w1-1 replay it is non-zero only during the two death/respawn sequences.

2. Super Mario Bros. 3 — mario3 (NES)

Object ids come from Level_ObjectID (\$0671, 8 slots), decoded through the 170-entry OBJ_* table generated from the disassembly. Verified: a World 1-1 replay decodes to Goomba, ParaGoomba, RedTroopa and VenusFireTrap — that level's actual roster.

Player

Event	Description	Detection
Player_damaged	Hit, losing the current suit.	player_suit_lost (the suit-lost poof counter) becomes non-zero, falling back to invisibility_timer (Player_FlashInv, the post-hit blink). Suppressed within 120 frames of a death.
Player_died/Enemy	Killed by an enemy.	player_is_dying (\$00F1) becomes 1 .
Player_died/Fall	Dropped off the bottom of the screen.	player_is_dying becomes 2 .
Player_died/Timeout	Time ran out.	player_is_dying becomes 3 .
Life_gained	Gained an extra life.	lives increases.

Event	Description	Detection
Player_state/{Small, Super, Fire, Raccoon, Frog, Tanooki, Hammer}	The suit the player wears, one row per continuous stretch. Together they tile the frames on which the player is alive in the level.	Maximal runs of a constant powerup (Player_Suit, \$00ED) over the alive frames (outside Screen/Death, Screen/Game_over, Screen/Map). A hit as Fire or in a suit drops the player to Super, so one row ends and the next starts on the same frame.
Player_state/Star	Star invincibility.	invincibility_timer (Player_StarInv) non-zero run, ~448 frames.
Player_state/Hit_recovery	Post-hit invulnerability while blinking, ~113 frames.	invisibility_timer (Player_FlashInv) non-zero run.
Player_state/Flying	Flying with a flight-capable suit.	flight_timer (Player_FlyTime) non-zero run.
Player_state/Statue, Player_state/Kuribo_shoe	Tanooki statue form; riding Kuribo's Shoe.	statue_timer / kuribo_shoe non-zero run. Both are rare (1 and 5 repetitions in the corpus).
P-Switch_started, P-Switch_expired	A P-Switch is active.	p_switch_timer (Level_PSwitchCnt) non-zero run.

player_is_dying states the cause of death outright. It replaces the previous timer-freeze plus X-position heuristic, and moved 942 repetitions from the catch-all failed/killed to failed/fall.

Objects

Event	Description	Detection
Enemy_on_screen/{type}	An enemy became visible.	A visible track opens in an object slot (Objects_State != 0 and both Objects_SprHVis and Objects_SprVVis clear); type from object_id_*.
Enemy_defeated/Stomp/{type}	Stomped.	object_state_* enters 7 (OBJSTATE_SQUASHED), or enters 3 (OBJSTATE_SHELLED) from a non-shell state — a stomp that shells an enemy.
Enemy_defeated/Projectile/{type}	Killed by a projectile or tail.	object_state_* enters 6 (OBJSTATE_KILLED) or 8 (OBJSTATE_POOFDEATH).
Shell_started_moving	A shelled enemy was kicked.	object_state_* enters 5 (OBJSTATE_KICKED).
Projectile_on_screen/{type}	A hammer, fireball, cannonball etc. became visible.	special_obj_id_* (SpecialObj_ID) non-zero, decoded through the 22-entry S0BJ_* table.

Event	Description	Detection
Item_on_screen/{1Up,Starman,Mushroom,FireFlower,SuperLeaf}	A power-up became visible.	Object ids 0x0B, 0x0C, 0x0D, 0x19, 0x1E.
Item_collected/Coin	Collected a coin.	coins_p1 increases, or wraps at 100.
Block_smashed	Destroyed a brick.	score increases by exactly 1 (10 points) while in_air is set.

Sixty object ids are excluded from Enemy_* as scenery, platforms, items or engine control objects. The exclusion list is derived from the disassembly's own OBJ_* names rather than hand-written, so it stays correct if the table is regenerated.

Level

Event	Description	Detection
Level_started	First attempt at this level in the run.	Frame 0 of a rep-000 file.
Level_restarted	A retry after dying.	Frame 0 of a rep-001+ file. In SMB3 one level attempt spans up to three one-life .bk2 files.
Goal_card_visible	The end-of-level roulette card is on screen, 0.5-4 s before it is taken. Untyped: its face cycles until touched.	A visible track of object id 0x41 (OBJ_ENDLEVELCARD). An earlier release keyed on the card items 0x21-0x23 (PowerUpMushCard etc.), which never appear at a level end, so the event never fired.
Level_completed	Finished the level.	A goal card enters the inventory (goal_cards_p1_{1,2,3} increases).
Goal_card_collected/{Mushroom,Flower,Star,...}	Which card was collected.	The same transition; the card value names it.
Auto_scroll_started	The level began scrolling on its own.	level_hautoscroll (Level_HAutoScroll) becomes non-zero.

3. Shinobi III — shinobi (Genesis)

This dataset is materially less well characterised than the other three, and the document should be read with that in mind. There is no public RAM map for Shinobi III, and roughly half the entries in its data.json are background/palette scratch (backTree, backbrush, ...) with no gameplay meaning. Everything below is derived from a counter whose meaning is unambiguous.

Event	Description	Detection
Player_damaged	Hit; the health bar decreased.	health decreases. Suppressed near a death, since dying drains the bar.
Health_gained	Collected a health item.	health increases.
Player_died	Lost a life. Durational.	lives decreases; onset is walked back to the frame health reached 0 (up to 400 frames earlier) and duration runs to the life loss, since lives only drops once the death animation and fade have played, 2-6 s after the fact.
Life_gained	Gained a life.	lives increases.

Event	Description	Detection
Enemy_defeated	Defeated an enemy. Untyped.	instantScore increases by one of the documented enemy values: 200 (basic enemies), 300 (mortars, machineguns), 400 (cauldron-heads), 500 (anti-riot cop, hovering ninja).
Projectile_appeared/Shuriken	Threw a shuriken.	shurikens decreases by exactly 1.
Item_collected/Shurikens	Picked up shurikens.	shurikens increases. The amount is not carried on the row; it is recoverable from shurikens in the repetition's <code>_variables.json</code> .
Weapon_powerup_started/Ninjutsu{n}	Used a ninjutsu.	ninjitsu decreases; typeOfNinjitsu names it. Happens in only 8 of 666 repetitions, and typeOfNinjitsu is always 0, so ninjutsu is not tracked as a state.
Player_state/Hit_recovery	The game's post-hit counter, during which the player flashes.	Non-zero run of hit_timer (new , \$FF4165 -- the byte at raw offset \$FF4164 of the word-swapped work RAM) that begins within 3 frames of a Player_damaged: 80 or 64 → 0, one per frame. Runs from 48 (knock-backs costing no health) and the 25-frame idles at 1 are ignored. Cut where health reaches 0.
Player_state/Normal	Shinobi's single form; present so that every dataset has a form for every frame of play.	One row per stretch of frames on which the player is alive (outside Screen/Death and Screen/Game_over). The temporary weapon upgrade is not tracked.
Level_started	A gameplay attempt began.	Frame 0.
Level_completed	Finished the level.	The start of the end-of-level fade that closes the recording: the last blackScreen stretch of the repetition holds 22 or 62 (see Screens above). Over the corpus this is present in 519 of 536 cleared repetitions and in none of the 130 failed ones; the 17 misses are recordings that stopped inside the last enemy wave, before the fade.

Known limitations

- **No Enemy_on_screen / Enemy_counter.** Nothing in the available RAM map locates enemy objects.
- **Enemy_defeated is untyped**, and only the four documented score values are counted. Other increments occur — 1000 is common, and 250/350/700/3000/5000 appear — but nothing available attributes them, so they are deliberately not counted rather than guessed at. Roughly a third of scoring events are therefore not represented. (The previous pipeline counted only 200 and 300, silently dropping the 400- and 500-point kills its own docstring listed.)
- **Level_completed depends on the recording reaching the fade.** The previous pipeline fabricated one five seconds before the end of any repetition in which no life was lost — a property of the whole repetition rather than a moment in time. An earlier release of this module then dropped the event, having found blackScreen set in every repetition; it is, but that value (40) is the scroll lock of an enemy wave, and the *trailing* fade value discriminates perfectly (see the Level_completed row above). 17 cleared repetitions whose recording stopped inside the last enemy wave get no event; their `_summary.json` Outcome still says cleared.
- Player_died and Life_gained are **new**: the previous pipeline had no player-death event at all, despite lives recording it unambiguously.

4. Controller actions — all games

Not covered by the original specification; normalised across games so the same button role carries the same label, with the raw button preserved in the **button** column. **duration** spans the hold.

Event	NES	SNES	Genesis
Action/Left, Action/Right, Action/Up, Action/Down	D-pad	D-pad	D-pad
Action/Jump	A	B, A	C
Action/Run	B	Y, X	—
Action/Attack	—	—	B
Action/Ninjutsu	—	—	A
Action/Start	START	START	START
Action/Select	SELECT	SELECT	MODE
Action/Other	—	L, R	X, Y, Z

Action/Other covers buttons with no documented function in the game; the previous pipeline dropped them entirely (the SNES L button is pressed 30 times in a single sampled replay).

5. The container row

Event	Description	Detection
gym-retro_game	One repetition of gameplay — one .bk2 file.	Taken from the run's plain_events.tsv. The only row that carries stim_file; every other event falls inside its window. duration is the replay's frame count divided by the true frame rate. It sorts first among rows sharing its onset, so slicing between container rows assigns every event to the right repetition.

Repetition outcomes

Not events, but recorded per repetition in gamelogs/*_summary.json under Outcome:

Label	Meaning
cleared	The level was finished.
failed/killed	The final death was caused by contact.
failed/fall	The final death was a fall off the screen.
failed/timeout	The timer ran out.
incomplete/warp	The player left through a warp zone.
incomplete/interrupted	The recording stopped mid-level.

For Super Mario Bros. the label describes how the repetition *ended*, so when several lives are lost only the final death is classified. The previous logic tested `jump_airborne == 3`, which is the flagpole slide **and** the vine climb, so warps taken up a vine were labelled cleared; 71 mario and 59 mariostars repetitions changed label when this was corrected. For SMB3 the cause comes from `player_is_dying`, which dropped the catch-all failed/killed from 91% to 67% of repetitions. Shinobi's outcome remains "was a life lost", the only signal its RAM map supports.

Provenance

RAM addresses and value tables are taken from published disassemblies and each was verified against an independent signal before being used:

- **Super Mario Bros.** — [1wErt3r's SMBDIS.ASM](#)

- **Super Mario Bros. 3** — [captainsouthbird's disassembly](#)
- **Super Mario All-Stars** — [Data Crystal's partial RAM map](#), plus the empirical finding that its object ids match SMB1's verbatim
- **Shinobi III** — no public RAM map exists

Independent checks on the generated events: the schema and controlled vocabulary (V0), invariants recomputed from the raw RAM by a different route than the generator (V1), and cross-port agreement between the NES and SNES versions of Super Mario Bros. (V2), where 15 of the 21 levels containing enemies yield identical rosters on the two consoles.

What none of that establishes is agreement with what is actually on screen. That needs the video review in `videogames_utils.events.review`, which samples events, cuts labelled clips from the frame-aligned recordings, and reports precision and recall per event type. Until it has been run, the accuracy figures quoted here are about internal consistency, not about ground truth.